



AI DIAL Chat

A modular chat application, a backend edge, and reusable workspace libraries.

4 applications

28 libraries

33 technical sessions

Source snapshot 23fa39459d89 · 2026-09-21



Start with the user workflows

01

Converse with context

Choose a deployment, attach files, send messages and inspect evidence.

02

Reuse knowledge

Browse prompts and skills, then apply them to a conversation.

03

Manage shared work

Share or publish resources and inspect scheduled task runs.



Four apps have different responsibilities

Application	Responsibility
chat	React SPA and application integration
chat-api	NestJS BFF, auth, persistence and Core access
chat-overlay-sandbox	Browser host for overlay integration cases
mcp-app-sandbox	Separate-origin sandbox proxy for MCP UI



Libraries for the conversation workspace

01

Conversation

conversation-input,
conversation-messages,
conversation-panel,
conversation-stages

02

Evidence

attachment-input,
attachment-canvas,
source-panel,
quotations

03

Navigation

navigation-panel,
sidebar, starter-buttons,
settings-panel



Libraries for knowledge and integration

01

Author and discover

catalog, builder-form, prompt-editor, prompts, skill-editor, skills, toolset-editor

02

Automate and distribute

scheduled-tasks, share, publish-panel, usage-dashboard

03

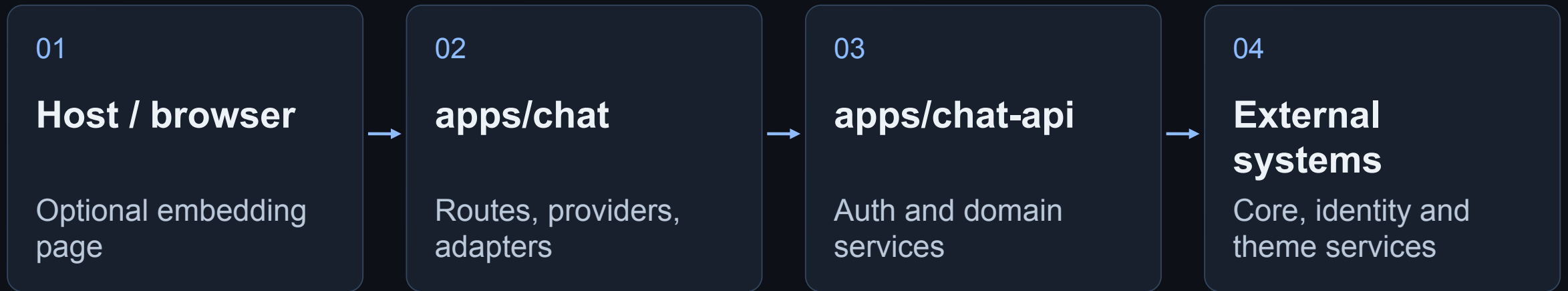
Integrate and share code

chat-api-client, chat-hooks, chat-shared, chat-overlay, mcp-apps



Application edges connect the layers

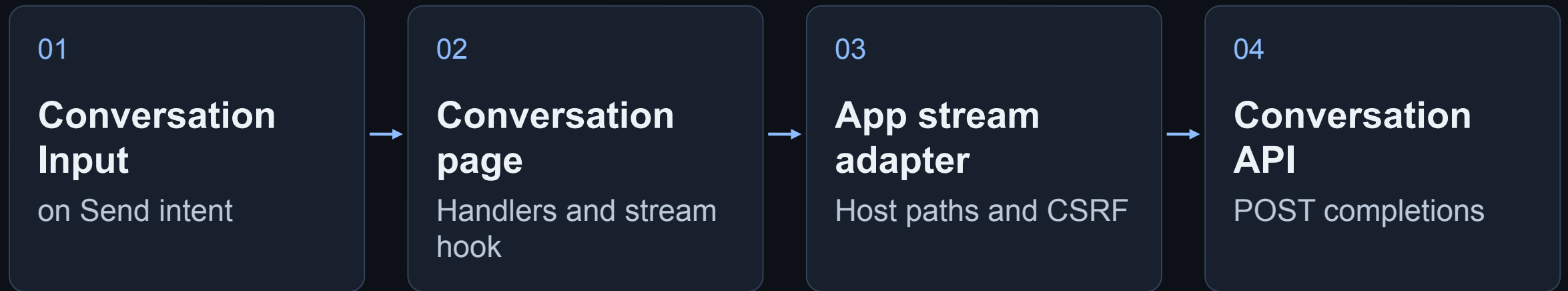
A simplified ownership map, not a complete dependency graph.



UI libraries receive data and behavior through public contracts.



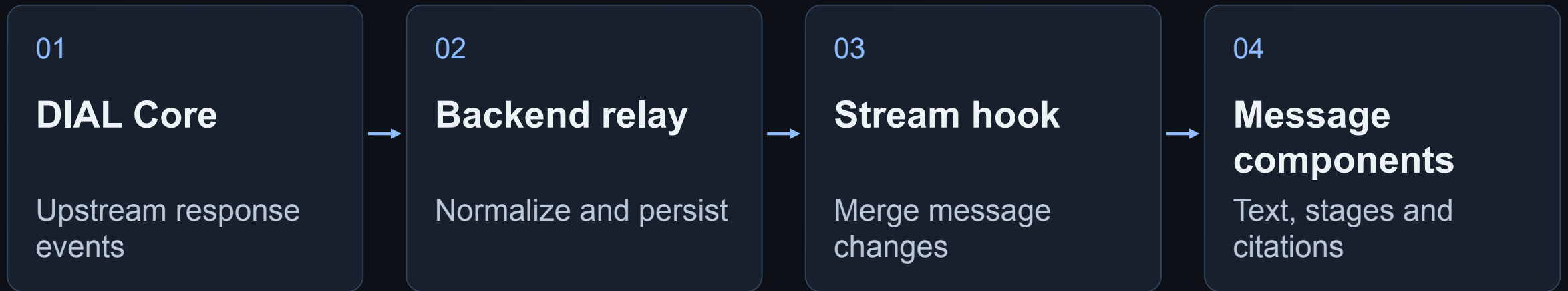
One message enters the backend



`POST /api/v1/conversations/completions`



Chunks return through a shared lifecycle



Closing the browser connection is different from explicitly stopping generation.

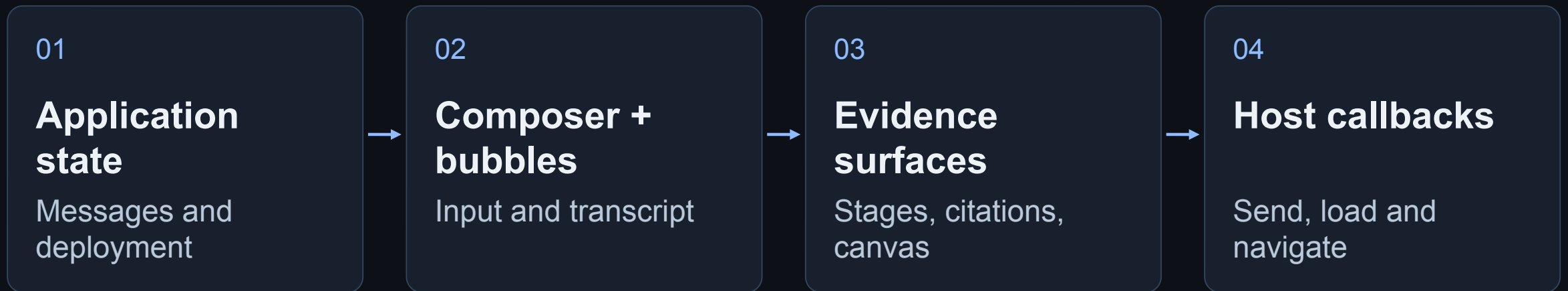


Separate clients, hooks and adapters

Layer	Concrete role
Generated API client	Fetch operations and DTO serialization
Headless hooks	Request/UI lifecycles with injected dependencies
Application adapters	Client configuration, CSRF, origin and route mapping
UI libraries	Render state and emit typed user actions



Compose a view from focused parts



A library can be adopted without copying the whole route implementation.



See three libraries in one host

Reproducible demo · actual workspace source components



DEMO · ACTUAL WORKSPACE COMPONENTS

Inspect an assistant workspace

Synthetic data · local source snapshot · no backend calls



Compose a screen

The host selects rows, formats amounts and supplies stage data. Each library renders its own part.

DEMO · real components rendered from this source snapshot



Choose a reuse boundary

01

Reuse a component

Bring typed data, callbacks, labels and its stylesheet.

02

Build a custom UI

Use chat-hooks and configured generated operations.

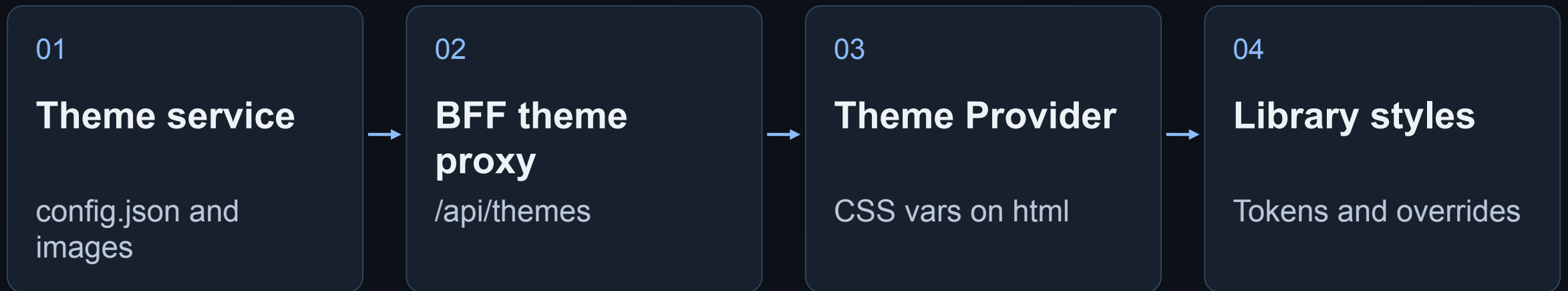
03

Embed the full app

Use ChatOverlay with an existing chat deployment.



Theme tokens travel from host to component



Theme selection is app-owned; reusable libraries consume resolved styling.



Localization support has a clear current limit

01

English resources

The current i18n initialization registers english translations.

02

RTL switching

ar, he, fa and ur set document direction to rtl.

03

Host-localized libraries

Labels and logical CSS keep language ownership at the edge.



Authentication belongs at the BFF



Configured clients and session details stay outside ordinary UI libraries.



Two upstream APIs share the browser stream

Server flag: `RESPONSES_API_ENABLED` · deployment feature: `responsesApi`

Server flag and deployment	Upstream path
Server flag = false	Chat Completions
Flag true; capability missing/false	Chat Completions
Flag true; responsesApi = true	Responses adapter



Run the applications locally

From the repository root, after configuring backend environment values.

```
npm ci
npm run start:all

# Frontend: http://localhost:4207
# Backend:  http://localhost:5000

npm exec nx serve chat-overlay-sandbox
npm run start:mcp-app-sandbox
```

Use a Node version compatible with the locked Nx/Vite stack.



Use the checks that match the change

Purpose	Repository command
Types, lint and tests	<code>npm run verify:full</code>
Build the apps/libraries	<code>npm run build:all</code>
Check docs and manifests	<code>npm run validate:docs</code>
Check generated client drift	<code>npm run openapi:check</code>



Modularity gives specific engineering benefits

01

One editor field set

builder-form validation
and fields are reused by
multiple editors.

02

Reusable evidence UI

The same canvas
displays several typed
content kinds.

03

Replaceable host behavior

Callbacks let a host
choose its own routing
and persistence.



Tradeoffs remain visible

01

Integration is real work

A host must supply data, labels, adapters and the correct peer dependencies.

02

Boundaries have drift

Some current libs import broader packages than the written rules permit.

03

Runtime state is local

Generation and several caches rely on process/browser memory.



Start with a vertical path through the code

01

Understand the assembly

Read main.tsx, app.tsx and one conversation or editor route.

02

Trace its contract

Follow a component prop into the host adapter and backend domain.

03

Reuse one piece

Open that library deck, its full example and its source references.